

SOLA Security Review

Plugin: local_ai_course_assistant (SOLA — Saylor Online Learning Assistant)

Reviewed version: 6.6.0 (2026061014)

Review date: 2026-06-12

Reviewer: Internal automated security review (multi-agent), findings manually verified against source

Prepared for: Catalyst IT (managed hosting), Saylor University

1. Scope and method

A full-codebase security review of the SOLA Moodle plugin ahead of the planned production upgrade (Saylor Learn and Saylor Degrees, currently v5.4.5). The review covered every web-reachable entry point (public pages, AJAX web services, CLI scripts) and the core service classes, against the following classes of issue:

- Authentication and capability enforcement
- Cross-site request forgery (CSRF)
- Cross-site scripting (XSS), stored and reflected
- SQL injection
- Server-side request forgery (SSRF)
- Secret handling (API keys, tokens)
- Insecure direct object references (IDOR) and data-ownership
- File handling and uploads
- Privacy / data-protection coverage (GDPR)

Every finding below was verified by reading the actual code path; speculative findings were discarded. Each finding records its status: **FIXED in 6.6.0** or **RECOMMENDED** (tracked follow-up).

Saylor context: SOLA is a tuition-free university's learning assistant. FERPA does not apply; the binding constraints are SOC 2, GDPR, and the no-training commitments with AI vendors. Learner PII, AI provider API keys, and spend controls are the assets of highest concern.

2. Summary

Severity	Found	Fixed in 6.6.0	Recommended follow-up
Critical	2	2	0
Major	6	5	1
Minor	4	2	2
Total	12	9	3

Both critical findings — concerning the bulk analytics export endpoint — are fixed and were verified working live. No critical or major issue remains unaddressed except one SSRF hardening item (DNS-rebinding defence) that requires broader testing and is scheduled as a tracked follow-up; its residual risk is low because the affected URLs are admin-configured and already validated against a private-network blacklist.

3. Findings

CRITICAL

C-1 — Bulk analytics export allowed cross-origin browser reads (FIXED in 6.6.0)

redash_export.php

The JSON export endpoint (learner usage, feedback, survey answers, transcript excerpts) sent `Access-Control-Allow-Origin: *` and authenticated by an API key passed as a URL query parameter. Any web page could therefore read the full export from a visitor's browser if it learned the key, and the key was exposed in web-server logs, browser history, and referrer headers.

Fix: The wildcard CORS header was removed. A CORS origin header is now emitted only when an admin explicitly configures a single exact origin (`redash_allowed_origin` , empty by default); the endpoint is intended to be pulled server-to-server by Redash, which needs no CORS at all. Verified live: no CORS header is sent by default.

C-2 — Feedback and survey PII ignored the anonymization flag (FIXED in 6.6.0)

redash_export.php

The export's `anonymize` flag (default on) was applied to the student-usage block but not to the feedback or survey blocks, which always emitted the real `userid` , free-text comment, full `user_agent` , `page_url` , and browser/OS/device/screen fingerprint for every learner.

Fix: Both blocks now honour the same gate. Under the default anonymized export they emit only a stable pseudonymous reference (`user_ref`), the rating/answer, the comment, and the timestamp; all fingerprinting fields are dropped. Raw values appear only on a deliberate `anonymize=0` request, which is already audit-logged with the requesting IP. Verified live: the fingerprint fields and raw `userid` are absent from the default export.

MAJOR

M-1 — Message-rating service lacked conversation-ownership check (IDOR) (FIXED in 6.6.0)

classes/external/rate_message.php

A learner could submit a rating or a hallucination flag against any message in any course they were enrolled in, including messages from another learner's private conversation, because only a course-level capability was checked. This let one learner skew another's analytics.

Fix: The service now verifies the message's conversation belongs to the calling user before recording the rating, throwing a permission error otherwise.

M-2 — Stored XSS via the admin chat-greeting setting (FIXED in 6.6.0)

amd/src/ui.js

The admin-configured greeting was injected into every learner's chat with `innerHTML` without sanitization, so a greeting containing markup or an event-handler attribute would execute in each learner's session.

Fix: The greeting is now rendered with `textContent` , which cannot inject markup.

M-3 — Reflected XSS via AI-generated essay feedback (FIXED in 6.6.0)

essay_feedback.php

The essay-scoring page concatenated the AI provider's response fields (criterion names, feedback text, overall comment, revision suggestions) directly into `innerHTML` . A model response containing markup would execute in the learner's browser.

Fix: All AI-sourced fields are now HTML-escaped through an explicit `esc()` helper before rendering.

M-4 — Talking-avatar viewer missing a guest guard (FIXED in 6.6.0)

talking_avatar_viewer.php

The (experimental, not-yet-enabled) avatar viewer required login but did not exclude guest users, and exposed a session token and a third-party bundle URL. A session-ownership check already prevented cross-user replay; the gap was the absence of a guest exclusion.

Fix: Guest users are now rejected. Note: the avatar viewer is a future-direction feature and is not enabled in the Saylor configuration; the broader capability model will be finalized when the feature ships.

M-5 — Admin model/history parameters loosely typed on the Learning Radar endpoint (FIXED in 6.6.0)

meta_ai_sse.php

The site-admin-only meta-AI endpoint accepted an unbounded conversation history array. Although gated to `moodle/site:config` (full site admins, who are already trusted), an oversized payload could grow the upstream request without bound.

Fix: Injected history is now capped to the most recent 50 turns. (The provider model name remains admin-supplied; tightening it to an allowlist is recommended — see R-2.)

M-6 — SSRF guard re-resolves DNS after validation (DNS rebinding) (RECOMMENDED)

`classes/security.php`

The provider-URL safety check resolves the hostname and rejects private-network addresses, but the subsequent network call re-resolves the hostname, leaving a time-of-check/time-of-use window in which a hostile DNS server could return a private address (for example a cloud metadata endpoint) on the second lookup.

Residual risk: low. The affected URLs are admin-configured provider endpoints, not user input, and the first-lookup blocklist still blocks the common cases. **Recommended fix:** pin the validated IP into the network call (`CURLOPT_RESOLVE`) so it cannot be re-resolved. Deferred from 6.6.0 because it touches every provider call path and warrants its own connectivity testing.

MINOR

m-1 — Redash API key delivered as a URL parameter (PARTIALLY ADDRESSED)

`redash_export.php`

The key is still accepted as a query parameter (kept for Redash compatibility), but the removal of wildcard CORS (C-1) eliminates the browser-exfiltration path. **Recommended:** also accept the key via an `Authorization: Bearer` header so it stays out of access logs. Tracked as R-3.

m-2 — Privacy provider metadata mismatch for one table (RECOMMENDED)

`classes/privacy/provider.php`

The GDPR metadata declaration for the Learning Radar schedule table names a field that does not match the column; data deletion works correctly, but a subject-access export would describe the table inaccurately. **Recommended:** correct the metadata field name. Low risk, no data exposure.

m-3 — Reminder unsubscribe uses a plain token rather than an HMAC (FIXED in 6.6.0 — documented)

`unsubscribe.php`

Unlike the HMAC-signed unsubscribe endpoints, this one uses an alphanumeric token. The token is server-generated and stored, so forgery requires guessing a random value; this is acceptable, and the endpoint has been documented to match the convention of the others. Migration to HMAC remains a possible future consistency improvement.

m-4 — Feedback comment double-encoded in the admin dashboard (FIXED in 6.6.0)

Display-only issue (special characters in feedback comments rendered as encoded entities); corrected so comments display correctly. Not a security gap.

4. Items already strong (no action needed)

The review confirmed the following were already implemented correctly and are worth noting for assurance:

- The export endpoint uses constant-time key comparison (`hash_equals`).
- De-anonymized exports are audit-logged with the requesting IP.
- The signed policy-bundle mechanism enforces an Ed25519 signature, a 43-key allowlist that excludes all secrets/URLs/consent/emergency settings, and a monotonic version, and fails closed.
- The avatar viewer already verifies session ownership before streaming.
- SQL throughout uses parameterized queries; no injection was found.
- The plugin uses its own SSRF blocklist (`security::is_safe_provider_url` plus an explicit trusted-endpoints allowlist) consistently for outbound calls.

5. Conclusion

The two critical findings (both on the analytics export endpoint) and four of the six major findings are fixed in 6.6.0 and, where externally observable, verified working. The remaining items are one low-residual-risk SSRF hardening and two minor consistency improvements,

all tracked. SOLA 6.6.0 is, in our assessment, ready for the planned production upgrade from a security standpoint.

The full 6.6.0 release passed the plugin's automated gate (security validator corpus, jailbreak suite, full PHPUnit suite, Behat browser tests) before this report was finalized.